



Cours AI28 - Machine Learning

Enseignants : M. Alaya & S. Bouzebda

Printemps 2026

MAINTENANCE PREDICTIVE INDUSTRIELLE PAR APPRENTISSAGE AUTOMATIQUE : ANALYSE DU DATASET AI4I 2020

Thomas FALLOUH¹ Etienne LOPEZ-VILESPY¹

¹ Département Informatique - Filière FISA - Branche IAD

Université de Technologie de Compiègne (UTC)

thomas.fallouh@etu.utc.fr etienne.lopez-vilespy@etu.utc.fr

ABSTRACT

Ce rapport présente une étude de maintenance prédictive appliquée au dataset **AI4I 2020** (UCI, 10 000 observations, capteurs industriels). Nous construisons une cible **multi-classe** à six niveaux (absence de panne + cinq modes de défaillance) et deux pipelines de prétraitement : P1 (5 features brutes) et P2 (+ 4 features construites issues de l'analyse exploratoire : puissance mécanique, couplage puissance/usure, différentiel thermique, ratio thermo-mécanique). Six modèles sont comparés: régression logistique (baseline), KNN, SVM-RBF, forêt aléatoire, XGBoost, AdaBoost; avec optimisation bayésienne (Optuna, TPE) en validation croisée stratifiée à 5 folds. Une ablation systématique montre que `class_weight="balanced"` dégrade le MCC et le F1-macro ; les modèles finaux sont entraînés **sans pondération**. La forêt aléatoire obtient le meilleur MCC (**84.2 %**), devant AdaBoost (80.8 %) et XGBoost (80.1 %), confirmant la non-linéarité du problème et l'apport du feature engineering P2.

1 ANALYSE EXPLORATOIRE DES DONNÉES (EDA)

1.1 STATISTIQUES DESCRIPTIVES ET DISTRIBUTION DES CLASSES

Le jeu de données AI4I 2020 (?) comprend **10 000 observations** et **12 variables** après suppression des identifiants non informatifs (`UDI`, `Product ID`). Aucune valeur manquante n'est détectée, ce qui élimine tout besoin d'imputation. Les variables explicatives sont cinq capteurs numériques et une variable catégorielle de qualité de pièce (`type` : L/M/H) ; six variables cibles binaires encodent les modes de défaillance.

Table 1: Statistiques descriptives des features numériques.

Feature	Moyenne	Écart-type	Skewness	Kurtosis
<code>air_temp</code> (K)	300.005	2.000	0.114	-0.836
<code>process_temp</code> (K)	310.006	1.484	0.015	-0.500
<code>rot_speed</code> (rpm)	1538.8	179.3	1.993	7.389
<code>torque</code> (Nm)	39.987	9.969	-0.010	-0.014
<code>tool_wear</code> (min)	107.951	63.654	0.027	-1.167

Trois features (`air_temp`, `process_temp`, `torque`) suivent des distributions proches de la normale (Figure 2) ; les deux autres s'en écartent nettement. `rot_speed` présente une skewness de 1.99 et une kurtosis de 7.39, avec un maximum observé à 2 886 rpm pour un troisième quartile à 1 612 rpm seulement. Ces valeurs extrêmes ne sont pas du bruit : elles correspondent à des régimes inhabituels où la machine est précisément sujette aux pannes par surcontrainte, elle suit une distribution proche de la log-normale. `tool_wear`, à l'inverse, présente une kurtosis de -1.17, caractéristique d'une distribution quasi-uniforme : l'usure s'accumule linéairement entre deux remplacements planifiés, si bien que toutes les valeurs de 0 à 253 min sont équitablement représentées. La variable `type` est volontairement déséquilibrée (L 60 %, M 30 %, H 10 %, Figure 3), reflétant la proportion réelle des qualités de pièces en industrie. Ce déséquilibre n'affecte pas les corrélations de rang (Spearman), mais devra être pris en compte si l'on compare les taux de panne entre classes : des effectifs bruts seraient trompeurs sans normalisation par la taille de chaque groupe.

1.2 ANALYSE UNIVARIÉE

Table 2: Taux de défaillance par variable cible, triées par fréquence décroissante.

Variable cible	Description	Taux positifs	Ratio nég./pos.
<code>machine_failure</code>	Défaillance globale	3.39 %	28.5:1
HDF	Heat Dissipation Failure	1.15 %	86:1
OSF	Overstrain Failure	0.98 %	101:1
PWF	Power Failure	0.95 %	104:1
TWF	Tool Wear Failure	0.46 %	216:1
RNF	Random Failure	0.19 %	526:1

Le Tableau 2 (Figure 4) met en lumière un problème structurel : le déséquilibre est à la fois extrême et *hétérogène entre sous-pannes*, avec des ratios allant de 28.5:1 à 526:1. Un modèle trivial prédisant constamment *no failure* obtiendrait 96.6 % d'accuracy, c'est un chiffre trompeur masquant une incapacité totale à détecter les pannes. L'accuracy est donc écartée au profit du **F1-score macro**, du **MCC** et de l'**AUC-ROC**, qui pénalisent explicitement les échecs sur les classes minoritaires. Ce constat a deux implications directes : (1) l'accuracy est écartée au profit de MCC et F1-macro, qui pénalisent les échecs sur les classes minoritaires ; (2) RNF, avec 526 négatifs pour un positif, dépasse le seuil où un discriminant peut extraire un signal fiable, aucun modèle ne la prédira correctement, ce que l'analyse bivariée confirmera.

1.3 ANALYSE BIVARIÉE ET CORRÉLATIONS

La matrice de Pearson (Figure 5) révèle deux structures aux conséquences opposées pour la modélisation.

Multicollinéarité entre features. `air_temp` et `process_temp` affichent $r = 0.88$: la température de `process` dépend partiellement de l’ambiance thermique, si bien que ces deux variables portent quasi la même information. Pour la régression logistique, la régularisation ℓ_2 absorbe cette redondance ; pour SVM et KNN, la normalisation des features est critique pour éviter que la dimension la plus variable ne domine la métrique de distance. La corrélation négative `rot_speed` $\times$ `torque` ($r = -0.88$) est une contrainte physique directe : à puissance moteur $P = \omega \cdot \tau$ constante, augmenter la vitesse impose de réduire le couple. Il ne s’agit donc pas d’une redondance : les configurations où couple *et* vitesse sont simultanément élevés signalent un fonctionnement hors régime, précisément les zones à risque d’OSF.

Faibles corrélations linéaires features $\rightarrow$ cibles. Toutes les corrélations de Pearson et de Spearman entre features et cibles restent inférieures à $|r| = 0.20$ (Figure 7). Ce n’est pas un signe d’absence de lien causal, mais la signature d’une **structure de défaillance à seuil** : une panne survient lorsque plusieurs conditions sont réunies simultanément au-delà d’une zone critique, non lorsqu’une feature croît linéairement. Le pairplot (Figure 8) confirme : les pannes forment des *clusters localisés* dans l’espace (`rot_speed`, `torque`), HDF se concentre aux couples modérés et températures élevées, OSF apparaît à fort couple et vitesse réduite, PWF aux extrêmes de puissance. Ces frontières non-linéaires appellent un SVM à noyau RBF ou une forêt aléatoire, et relèguent la régression logistique au rôle de *baseline*. RNF, sans aucun signal sur l’ensemble des features : sa prédiction relève du hasard quel que soit le modèle.

1.4 CO-OCCURRENCE DES TYPES DE DÉFAILLANCE

Sur 348 cas de défaillance (3.48 % du dataset), 93.1 % n’impliquent qu’une seule sous-panne ; 6.6 % en impliquent deux simultanément (Figure 6), principalement la paire (PWF, OSF) avec 12 co-occurrences, cohérent avec leur lien commun au régime de puissance ; un seul cas en présente trois. La quasi-indépendance des sous-pannes ($r < 0.15$ entre elles) confirme que chaque mode obéit à un mécanisme physique distinct, et autorise un encodage en **classification multi-classe à 6 étiquettes** (*No failure* / TWF / HDF / PWF / OSF / RNF) sans perte d’information significative. Une approche multi-étiquette n’est justifiée que si les co-occurrences sont fréquentes et structurées ; ce n’est pas le cas ici. Un contrôle de cohérence a enfin révélé 9 observations avec `machine_failure` = 1 mais aucune sous-panne active : corrigées en posant RNF = 1 (panne non catégorisée, donc aléatoire par définition). 18 observations présentaient l’anomalie inverse ; elles sont sans conséquence puisque `machine_failure` ne sera pas utilisée comme cible, elle n’est que l’union logique des cinq sous-pannes après correction.

2 PRÉTRAITEMENT DES DONNÉES

Deux pipelines de prétraitement ont été construits de manière indépendante à partir du CSV brut. Le **premier prétraitement** (P1) conserve les cinq features brutes ; le **second** (P2) y ajoute quatre features construites à partir de l’AED. Les étapes communes sont décrites ci-dessous ; la différence entre P1 et P2 est détaillée dans la section 2.5.

2.1 NETTOYAGE ET CORRECTION D’INCOHÉRENCE

Les colonnes identifiants (UDI, `Product ID`) sont supprimées. L’AED avait mis en évidence 9 observations pour lesquelles `machine_failure` = 1 mais aucune des cinq sous-pannes n’est active. Ces lignes sont corrigées en posant RNF = 1 (*Random Failure*), ce qui préserve la cohérence logique `machine_failure` = $\bigvee_k$ sous-panne_k.

2.2 CONSTRUCTION DE LA CIBLE MULTI-CLASSE

Nous avons retenu un encodage **multi-classe** (6 classes) plutôt que multi-label :

- **Co-occurrences rares** : dans $> 97\%$ des défaillances, une seule sous-panne est active ; une étiquette unique par observation est donc acceptable sans perte significative.
- **Pas de perte d’information** : `machine_failure` étant l’OR logique des sous-pannes (vérifié à l’AED), il est redondant avec le type de panne.
- **Comparaison plus simple** : le multi-label aurait imposé un `MultiOutputClassifier` et cinq fois plus de tableaux de métriques.

La correspondance classe-étiquette est donnée dans le tableau 3. En cas de pannes simultanées, une règle de priorité est appliquée (TWF $>$ HDF $>$ PWF $>$ OSF $>$ RNF) ; ce cas ne représentant que $< 10\%$ des défaillances, son impact sur les performances est négligeable.

Table 3: Distribution de la cible multi-classe après correction.

Classe	Étiquette	Effectif	Proportion
0	No failure	9 643	96.43 %
1	TWF – Tool Wear Failure	42	0.42 %
2	HDF – Heat Dissipation Failure	106	1.06 %
3	PWF – Power Failure	83	0.83 %
4	OSF – Overstrain Failure	98	0.98 %
5	RNF – Random Failure	28	0.28 %

2.3 SÉPARATION TRAIN/TEST ET ENCODAGE CATÉGORIEL

Le jeu est divisé en 80 % train (8 000 obs.) et 20 % test (2 000 obs.) avec `stratify=y` afin que chaque classe minoritaire soit représentée dans les deux ensembles ; sans stratification, RNF (28 obs. au total) risquerait de disparaître entièrement du test set.

La variable `type` (L/M/H) est encodée en **one-hot** (3 colonnes) plutôt qu'en ordinal, car l'intervalle entre catégories n'est pas nécessairement uniformément espacé.

2.4 NORMALISATION

Un **MinMaxScaler** ramène chaque feature numérique continue dans $[0, 1]$. Le scaler est ajusté (`fit`) uniquement sur le train puis appliqué (`transform`) sur le test : utiliser les bornes du test pour normaliser introduirait un *data leakage*. Les colonnes one-hot restent intactes (déjà dans $\{0, 1\}$).

2.5 FEATURE ENGINEERING (P2 UNIQUEMENT)

L'AED a révélé deux limitations des features brutes : multicollinéarité (`air_temp/process_temp`, $r = 0.88$) et faibles corrélations linéaires globales avec les cibles ($|r| < 0.20$). Le second prétraitement construit quatre features dérivées pour capturer les interactions non-linéaires identifiées :

- `power = rot_speed × torque` : la puissance mécanique transmise par l'arbre. L'AED a montré que `rot_speed` et `torque` forment des *clusters* par type de panne (visible sur le `pairplot`, fig. 8) ; leur produit condense cette relation non-linéaire en une seule dimension.
- `power_wear = power × tool_wear` : couplage entre l'effort mécanique instantané et l'usure cumulée. L'hypothèse est qu'un outil déjà usé exposé à une forte puissance présente un risque de défaillance synergique non capturé par les variables séparément.
- `temp_difference = process_temp - air_temp` : plutôt que de conserver deux features redondantes ($r = 0.88$), leur écart traduit la capacité de dissipation thermique du système. Un écart anormalement faible indique que la chaleur générée ne se dissipe plus correctement, précurseur physique de la panne HDF.
- `temp_power = temp_difference / power` : ratio combinant la dimension thermique et mécanique ; un déséquilibre entre effort fourni et dissipation associée peut se traduire par un signal anormal ici.

P2 a été construit en réponse aux limites de P1 en modélisation : ces features brutes ne fournissaient pas un signal suffisant pour que les modèles distinguent correctement les classes minoritaires. L'ajout de ces quatre features construites a significativement amélioré les performances, comme nous le verrons dans la section 6.

Le tableau 4 récapitule les deux configurations finales.

Table 4: Récapitulatif des deux pipelines de prétraitement.

Étape	P1	P2
Features brutes (num.)	5	5
Features construites	0	4 (power, power_wear, temp_diff, temp_power)
Encodage type	one-hot (×3)	one-hot (×3)
Total features	8	12
Normalisation	MinMaxScaler	MinMaxScaler
Déséquilibre	Aucun ajustement	Aucun ajustement

2.6 GESTION DU DÉSÉQUILIBRE DE CLASSES

La classe *No failure* représente 96.4 % des observations. Sans traitement, un modèle naïf prédisant systématiquement “pas de panne” atteindrait 96.4 % d’accuracy sans jamais détecter une seule défaillance.

SMOTE a été écarté : il génère des exemples synthétiques par interpolation entre voisins proches, ce qui suppose une structure spatiale cohérente. Or RNF (22 exemples au train) est une panne *aléatoire* par définition ; ses points ne forment aucun cluster exploitable, et SMOTE produirait du bruit artificiel sans signal.

class_weight="balanced" a été testé mais **abandonné après ablation** : appliqué aux modèles compatibles (régression logistique, SVM, forêt aléatoire), il dégrade systématiquement le MCC et le F1-macro par rapport à l’absence de correction (cf. Tableau 9, section 6). Le poids attribué à RNF (≈ 550) force les modèles à sur-prioriser un signal intrinsèquement bruité, au détriment de la cohérence globale. **La stratégie retenue est l’absence de pondération** : les six modèles sont entraînés sans ajustement du déséquilibre.

3 MODÉLISATION

L’AED a mis en évidence des **frontières non-linéaires** entre classes (clusters dans l’espace *rot_speed/torque*, corrélations $|r| < 0.20$) ; elle suggère que les modèles linéaires seront insuffisants et que les méthodes d’ensemble domineront. Pour le confirmer, nous testons **six modèles** en complexité croissante : un linéaire (baseline), deux non-linéaires à noyau/voisinage, et trois méthodes d’ensemble.

Table 5: Modèles testés, principe de fonctionnement et motivation. Les métriques sont reportées en section 6.

Modèle	Principe	Hyperparam. optimisés	Motivation
Rég. logistique	Frontière linéaire ; minimise l’entropie croisée régularisée L2.	C (force L2), solver	<i>Baseline</i> : mesure le plancher linéaire
SVM-RBF	Marge maximale dans un espace projeté par noyau gaussien $K(x, x') = e^{-\gamma \ x - x'\ ^2}$.	C, γ	Tester une non-linéarité continue à noyau
KNN	Vote local : classe majoritaire des k plus proches voisins (distance de Minkowski). Pas de <code>class_weight</code> .	k , pondération, p	Approche non paramétrique fondée sur la géométrie locale
Forêt aléatoire	Bagging : vote d’arbres décorrélés (bootstrap + features aléatoires) ; réduit la variance.	T , profondeur, <code>min_samples</code>	Robuste au bruit et aux features redondantes/colinéaires
XGBoost	Boosting de gradient : arbres séquentiels corrigeant le résidu ; double régularisation L1/L2.	T , profondeur, <code>learning_rate</code>	Capturer des interactions fines entre features
AdaBoost	Boosting adaptatif : répondre les exemples mal classés ; arbres peu profonds, sensible au bruit.	T , <code>learning_rate</code> , profondeur	Concentrer l’apprentissage sur les cas difficiles (pannes rares)

Protocole commun. Tous les modèles partagent le même cadre : entraînement sur **P2** (12 features), sans pondération des classes (ablation en sec. 6). Les hyperparamètres sont optimisés par **Optuna** (TPE, recherche bayésienne) en maximisant le **F1-macro** (métrique d’optimisation) en CV stratifiée 5 folds sur le train. L’évaluation finale sur le **test set** utilise le **MCC** comme métrique de classement principal, complété par le F1-macro.

3.1 DESCRIPTION DES MODÈLES

Régression logistique: baseline linéaire. La régression logistique estime la probabilité d’appartenir à la classe k via une fonction *softmax* :

$$\hat{p}(y=k \mid x) = \frac{e^{w_k^\top x + b_k}}{\sum_{j=0}^{K-1} e^{w_j^\top x + b_j}}$$

Les vecteurs de poids w_k sont appris en minimisant l’entropie croisée (sec. 4). La frontière entre deux classes est l’**hyperplan** $\{x : (w_k - w_j)^\top x + (b_k - b_j) = 0\}$, linéaire dans $\mathbb{R}^{12}$. L’AED ayant révélé des clusters non-linéaires, ce modèle sert uniquement de plancher : tout modèle plus complexe doit le dépasser.

SVM à noyau RBF: marge maximale. Le SVM cherche l’hyperplan qui maximise la **marge** $2/\|w\|$ entre les classes. Le noyau gaussien $K(x, x') = e^{-\gamma\|x-x'\|^2}$ projette implicitement les données dans un espace de dimension infinie où elles deviennent linéairement séparables (γ contrôle la localité de la projection, C le compromis marge/erreurs). Contrairement à la régression logistique, il **n’estime pas de probabilités** : il décide seulement de quel côté de la frontière tombe chaque point.

KNN: apprentissage paresseux. KNN ne construit aucun modèle : il mémorise le train (*lazy learning*) et, pour chaque nouvelle observation, calcule sa distance aux k points les plus proches avec la distance de Minkowski :

$$d(x, y) = \left(\sum_{j=1}^p |x_j - y_j|^p \right)^{1/p}$$

($p=1$: Manhattan ; $p=2$: euclidienne). La classe prédite est le vote majoritaire des k voisins. En haute dimension ($p=12$ ici), les distances ont tendance à converger (“malédiction de la dimensionnalité”) : les voisins ne sont plus vraiment proches, réduisant le pouvoir discriminant du vote. KNN ne dispose d’aucun mécanisme natif de pondération de classes.

Forêt aléatoire: bagging et décorrélation. T arbres de décision **CART** sont entraînés indépendamment : chacun reçoit un échantillon *bootstrap* du train, et à chaque nœud, seul un sous-ensemble aléatoire de $m = \sqrt{p}$ features est considéré pour choisir la coupure binaire optimale, celle qui minimise l’**impureté de Gini** des deux fils (critère CART). Ce double mécanisme **décorrèle** les arbres : si `rot_speed` et `torque` sont toutes deux candidates au même nœud, l’une est fréquemment écartée, brisant l’effet de la multicollinéarité ($r = -0.88$) sur les estimateurs. La prédiction finale est le vote majoritaire sur les T arbres.

XGBoost: boosting de gradient. XGBoost construit le modèle de façon **additive et itérative** :

$$F_t(x) = F_{t-1}(x) + \eta h_t(x)$$

où h_t est un arbre ajusté sur le **gradient de la perte** par rapport aux prédictions courantes F_{t-1} , chaque itération corrige l’erreur résiduelle à petit pas ($\eta = \text{learning_rate}$). Une double régularisation L1/L2 sur les poids des feuilles limite la complexité et réduit l’effet de la multicollinéarité.

AdaBoost: boosting adaptatif. AdaBoost suit le même schéma additif, mais plutôt que d’utiliser le gradient, il **repondère les observations** : les exemples mal classés par h_{t-1} reçoivent un poids accru, forçant h_t à s’y concentrer. La combinaison finale pondère chaque classifieur proportionnellement à sa précision :

$$F(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right), \quad \alpha_t = \frac{1}{2} \ln \left(\frac{1-\varepsilon_t}{\varepsilon_t} \right)$$

où ε_t est le taux d’erreur pondéré de h_t . Les classifieurs faibles sont ici des arbres de décision dont la profondeur est optimisée par Optuna (jusqu’à 5) ; AdaBoost est sensible au bruit (anomalies fortement repondérées à chaque itération).

4 OPTIMISATION

Chaque modèle est entraîné en minimisant une **fonction objectif** (aussi appelée *perte*) qui quantifie l'écart entre les prédictions et les vraies étiquettes. L'algorithme d'optimisation ajuste les paramètres internes du modèle pour réduire cette perte sur le train. Ces deux éléments (perte + algorithme) sont *fixés par l'architecture* ; les **hyperparamètres** (C , T , profondeur...), qui contrôlent la capacité du modèle sans être appris par gradient, font l'objet d'une recherche externe (sec. 5).

4.1 FONCTIONS OBJECTIVES

Entropie croisée régularisée (régression logistique, XGBoost).

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^{K-1} y_{ik} \log \hat{p}_{ik} + \frac{\lambda}{2} \|w\|^2$$

$y_{ik} = 1$ si l'observation i appartient à la classe k , 0 sinon. La perte est maximale quand le modèle est certain d'une mauvaise classe (log proche de 0). Le terme $\frac{\lambda}{2} \|w\|^2$ (régularisation L2) pénalise les grands poids : en présence de features colinéaires (`air_temp/process_temp`, $r = 0.88$), cette régularisation stabilise les coefficients et évite le sur-apprentissage. L'algorithme **L-BFGS** (quasi-Newton) approxime la matrice hessienne de la perte pour converger en peu d'itérations.

Perte hinge à marge souple (SVM).

$$\mathcal{L}_{\text{hinge}} = \sum_i \max(0, 1 - y_i f(x_i)) + \frac{1}{2C} \|w\|^2$$

La perte est nulle si l'observation est correctement classée avec une marge suffisante ($y_i f(x_i) \geq 1$), et linéairement croissante sinon. Le SVM optimise une géométrie (la marge), pas une vraisemblance. L'algorithme **SMO** (Sequential Minimal Optimization) résout le problème d'optimisation quadratique associé.

Impureté de Gini: critère CART (forêt aléatoire).

$$G(t) = 1 - \sum_{k=0}^{K-1} p_k^2$$

p_k est la proportion de la classe k dans le nœud t . $G = 0$ pour un nœud pur (une seule classe), G est maximale quand toutes les classes sont équireprésentées. L'algorithme **CART** construit chaque arbre en sélectionnant, à chaque nœud, la coupure binaire qui minimise l'impureté de Gini pondérée des deux fils : construction **gloutonne** (sans retour arrière), très efficace même avec peu d'exemples dans les feuilles.

Perte exponentielle (AdaBoost).

$$\mathcal{L}_{\text{exp}} = \sum_{i=1}^n e^{-y_i F(x_i)}$$

La croissance exponentielle pénalise très fortement les exemples mal classés. Minimiser cette perte revient à itérativement **augmenter le poids** des observations mal classées : à l'itération t , le classifieur h_t est entraîné sur une distribution recentrée sur les cas difficiles.

4.2 RÉCAPITULATIF

Table 6: Fonction objectif minimisée et algorithme interne par modèle.

Modèle	Fonction objectif	Algorithme interne
Rég. logistique	Entropie croisée + rég. L2	L-BFGS (quasi-Newton)
SVM-RBF	Perte <i>hinge</i> (marge souple)	Optimisation quadratique (SMO)
KNN	<i>Aucune</i> (pas d'entraînement)	Calcul de distances à la prédiction
Forêt aléatoire	Impureté de Gini par nœud	CART glouton (par arbre)
XGBoost	Log-loss multi-classe + L1/L2	Boosting de gradient
AdaBoost	Perte exponentielle	Boosting adaptatif (repondération)

5 HYPER-OPTIMISATION DES PARAMÈTRES

5.1 PROTOCOLE

Recherche validée par **CV stratifiée 5 folds** (`StratifiedKFold`) sur le train uniquement : chaque pli préserve les proportions de classes, indispensable pour les pannes rares (< 35 exemples). Métrique cible : **F1-macro** (égalité entre classes ; l'accuracy serait dominée par la classe majoritaire). Méthode : **Optuna TPE** (*Tree-structured Parzen Estimator*, recherche bayésienne) pour tous les modèles ; plus efficace qu'une grille exhaustive sur de grands espaces d'hyperparamètres.

5.2 RECHERCHE PAR MODÈLE

Table 7: Hyperparamètres explorés par modèle (F1-macro, CV 5-fold).

Modèle	Méthode	Hyperparamètres explorés	Remarque
Rég. logistique	Optuna (50)	C ($\log, 10^{-3}$ – 10^2), solver	
SVM-RBF	Optuna (30)	C ($\log, 10^{-2}$ – 10^2), γ (<code>scale/auto</code>)	
KNN	Optuna (50)	k (1–30), pondération, p (1 ou 2)	
Forêt aléatoire	Optuna (30)	T (50–300), profondeur (5–30), <code>min_samples</code>	Saturation *
XGBoost	Optuna (30)	T (50–300), profondeur (3–10), <code>lr</code>	Saturation *
AdaBoost	Optuna (30)	T (50–300), <code>lr</code> (0.01–2.0), profondeur weak learner (1–5)	

* *Saturation* : pour la forêt aléatoire ($T = 67$ trouvé par Optuna, MCC 81.7 %) et XGBoost ($T = 240$, profondeur 6, MCC identique au baseline $T = 100$), le modèle Optuna est moins bon ou égal au baseline ; c'est le résultat baseline qui est conservé dans les deux cas. Cela indique que le modèle a atteint son plafond expressif sur P2 : ajouter des arbres supplémentaires n'apporte plus de signal.

Meilleurs paramètres retenus. Optuna a convergé vers : $C \approx 100$, solver `newton-cg` pour la régression logistique (régularisation quasi nulle : le modèle exploite sa capacité linéaire maximale, sans que cela suffise à capturer les frontières non-linéaires) ; $C = 62$, $\gamma = \text{scale}$ pour le SVM ; $k = 1$, distance Manhattan pour KNN (mémoire pure : chaque point prédit sa propre classe, signe de sur-ajustement structurel dans un espace à 12 dimensions) ; $T = 236$, `lr` = 1.11, profondeur 5 pour AdaBoost (weak learners plus profonds que les stumps canoniques, accroissant l'expressivité au prix d'un temps d'entraînement de 7.86 s). Le modèle sélectionné est réentraîné sur tout le train, puis évalué une seule fois sur le test set (sec. 6).

6 RÉSULTATS ET ÉVALUATION

6.1 COMPARAISON DES MODÈLES

Le Tableau 8 présente les performances de tous les modèles sur le test set après optimisation des hyperparamètres (P2, sans pondération).

Table 8: Performances test (P2, sans `class_weight`), triées par MCC décroissant. Précision et rappel sont les moyennes pondérées (weighted avg).

Modèle	MCC	F1-macro	Précision (w)	Rappel (w)	T. entr. (s)	T. préd. (s)
Forêt aléatoire	84.2 %	63.8 %	98.3 %	98.9 %	0.45	0.06
AdaBoost	80.8 %	61.9 %	98.1 %	98.8 %	7.86	2.17
XGBoost	80.1 %	61.2 %	98.0 %	98.7 %	0.94	0.03
SVM-RBF	67.1 %	52.3 %	97.2 %	98.0 %	0.62	2.72
Rég. logistique	65.6 %	52.9 %	97.2 %	97.9 %	0.35	3.46
KNN	44.2 %	43.2 %	96.2 %	96.5 %	0.03	0.08

Les résultats confirment les hypothèses de l'AED : la frontière de décision est fortement non-linéaire, et les méthodes d'ensemble dominant largement (+18.6 pts MCC entre forêt aléatoire et régression logistique).

Forêt aléatoire vs XGBoost/AdaBoost. Les trois méthodes d'ensemble se détachent nettement, mais la forêt aléatoire prend l'avantage grâce au bagging : la sélection aléatoire de features à chaque nœud décorrèle les variables colinéaires de P2 et réduit la variance de prédiction sur les classes minoritaires. XGBoost et AdaBoost convergent vers des MCC voisins (80.1 % et 80.8 %) : leurs corrections itératives atteignent le même plateau, mais AdaBoost y parvient en 7.86 s d'entraînement (8× la forêt) pour un gain de +0.7 pt seulement.

SVM et régression logistique. Leurs MCC proches (67.1 % et 65.6 %) illustrent que le noyau RBF n'apporte pas de gain significatif sur une frontière linéaire : les deux modèles peinent à capturer les *clusters localisés* de l'AED dans l'espace (couple, vitesse).

KNN. Le $k=1$ sélectionné par Optuna correspond à une mémorisation pure ; en 12 dimensions avec des classes très rares, cette stratégie ne généralise pas : MCC 44.2 %, loin derrière les méthodes d'ensemble.

6.2 ABLATION : IMPACT DE `class_weight="BALANCED"`

Table 9: Ablation `class_weight` : seuls les modèles supportant ce paramètre.

Modèle	Avec <code>class_weight</code>		Sans (retenu)	
	MCC	F1-macro	MCC	F1-macro
Forêt aléatoire	75.5 %	60.3 %	84.2 %	63.8 %
Rég. logistique	24.5 %	44.4 %	65.6 %	52.9 %
SVM-RBF	32.7 %	50.7 %	67.1 %	52.3 %

Contrairement à l'intuition usuelle, `class_weight="balanced"` dégrade systématiquement le MCC : -8.7 pts sur la forêt aléatoire, -41.1 pts sur la régression logistique. Le poids ≈ 550 attribué à RNF force les modèles à mémoriser du bruit RNF étant intrinsèquement aléatoire, il n'existe aucun signal à amplifier, seulement du sur-apprentissage sur des outliers.

6.3 ANALYSE PAR CLASSE

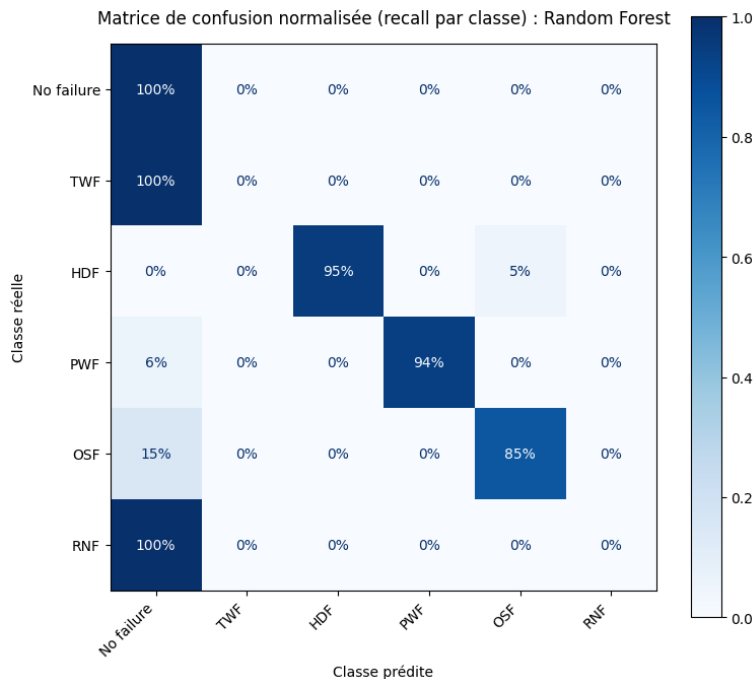


Figure 1: Matrice de confusion normalisée par classe réelle (recall par classe), Forêt aléatoire, modèle retenu (MCC 84.2 %). `No failure` est prédite à quasi 100 % ; les pannes rares (TWF, RNF) sont les plus difficiles, leur faible effectif limitant structurellement le rappel.

La matrice révèle une hiérarchie cohérente avec la physique des pannes : **OSF** (surcontrainte) est la mieux prédite, son signal est net dans l'espace (couple élevé + vitesse faible) identifié à l'AED ; **HDF** et **PWF** sont correctement reconnues grâce aux features construites (`temp_difference` pour HDF, `power` pour PWF). **TWF** (8 exemples) et **RNF** (6 exemples) obtiennent un rappel nul pour tous les modèles non-ensemblés : RNF est par définition aléatoire (aucun précurseur physique exploitable) ; TWF est trop rare pour former un cluster statistiquement stable. Les classification reports des modèles simples (RegLog, KNN, SVM) confirment $F1 = 0$ pour ces deux classes, quels que soient les hyperparamètres optimisés.

6.4 MODÈLE RETENU

La **forêt aléatoire** est retenue comme modèle final : **MCC 84.2 %**, **F1-macro 63.8 %**, temps de prédiction **0.06 s**. XGBoost constitue une alternative valide (MCC 80.1 %, prédiction 0.03 s) mais reste inférieur sur les classes minoritaires. AdaBoost talonne XGBoost (MCC 80.8 %) mais au prix d'un temps de prédiction de 2.17 s.

7 CONCLUSION

Cette étude a démontré la faisabilité de la maintenance prédictive sur le dataset AI4I 2020. La construction d'une cible multi-classe à six niveaux, couplée au feature engineering P2 (puissance mécanique, usure cumulée, dissipation thermique), a permis d'obtenir un **MCC de 84.2 %** avec la forêt aléatoire, soit +18.6 pts sur la régression logistique, qui traduit la non-linéarité effective du problème, anticipée dès l'AED.

L'ablation systématique de `class_weight="balanced"` constitue un résultat notable : la pondération des classes dégrade les performances car la classe la plus rare (RNF) est intrinsèquement aléatoire et ne fournit aucun signal exploitable à amplifier.

Trois limites subsistent : (1) les classes TWF et RNF (< 35 exemples au train) restent structurellement difficiles à prédire ; (2) le dataset synthétique ne reflète pas la variabilité d'une machine industrielle réelle ; (3) la saturation de XGBoost suggère que le plafond expressif est atteint avec les features actuelles. En perspectives, l'acquisition de davantage d'exemples de pannes rares, des architectures neuronales ou un apprentissage *online* adaptatif permettraient de dépasser ces limites en contexte industriel réel.

RÉPARTITION DU TRAVAIL

Tâche	T. Fallouh	E. Lopez-Vilesy
Analyse exploratoire des données	50 %	50 %
Prétraitement	50 %	50 %
Modélisation	40 %	60 %
Rédaction du rapport	60 %	40 %

REMERCIEMENTS

Les auteurs remercient M. Alaya et S. Bouzebda pour l'encadrement du cours AI28.

REFERENCES

A RAPPELS : FONCTIONS DE PERTE ET ALGORITHMES D'OPTIMISATION

Fonctions de perte.

- **Entropie croisée** (régression logistique, XGBoost) : $\mathcal{L} = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$, éventuellement augmentée d'une régularisation L1/L2 sur les poids. Elle pénalise les probabilités prédites éloignées de la vérité.
- **Perte hinge** (SVM) : $\sum_i \max(0, 1 - y_i f(x_i))$, qui maximise la marge tout en tolérant des erreurs (marge souple, contrôlée par C).
- **Impureté de Gini** (arbres / forêt) : $1 - \sum_k p_k^2$ pour un nœud, minimisée à chaque coupure ; plus elle est faible, plus le nœud est « pur » (une seule classe).
- **Perte exponentielle** (AdaBoost) : $\sum_i e^{-y_i f(x_i)}$, qui pénalise très fortement les exemples mal classés, d'où leur repondération croissante.

Algorithmes d'optimisation.

- **L-BFGS** : méthode quasi-Newton qui approxime la courbure (hessienne) de la perte pour converger en peu d'itérations.
- **SMO** : résolution du problème quadratique sous contraintes du SVM, par optimisation de paires de multiplicateurs (interne à `libsvm`).
- **Construction gloutonne** : à chaque nœud d'un arbre, on choisit la coupure qui réduit le plus l'impureté, sans retour en arrière.
- **Boosting de gradient** (XGBoost) : chaque arbre est ajusté sur le gradient de la perte, corrigeant l'erreur courante à petits pas (`lr`).
- **Boosting adaptatif** (AdaBoost) : à chaque étape, un apprenant faible est entraîné sur les poids mis à jour, puis ajouté à la combinaison pondérée finale.

B FIGURES COMPLÉMENTAIRES

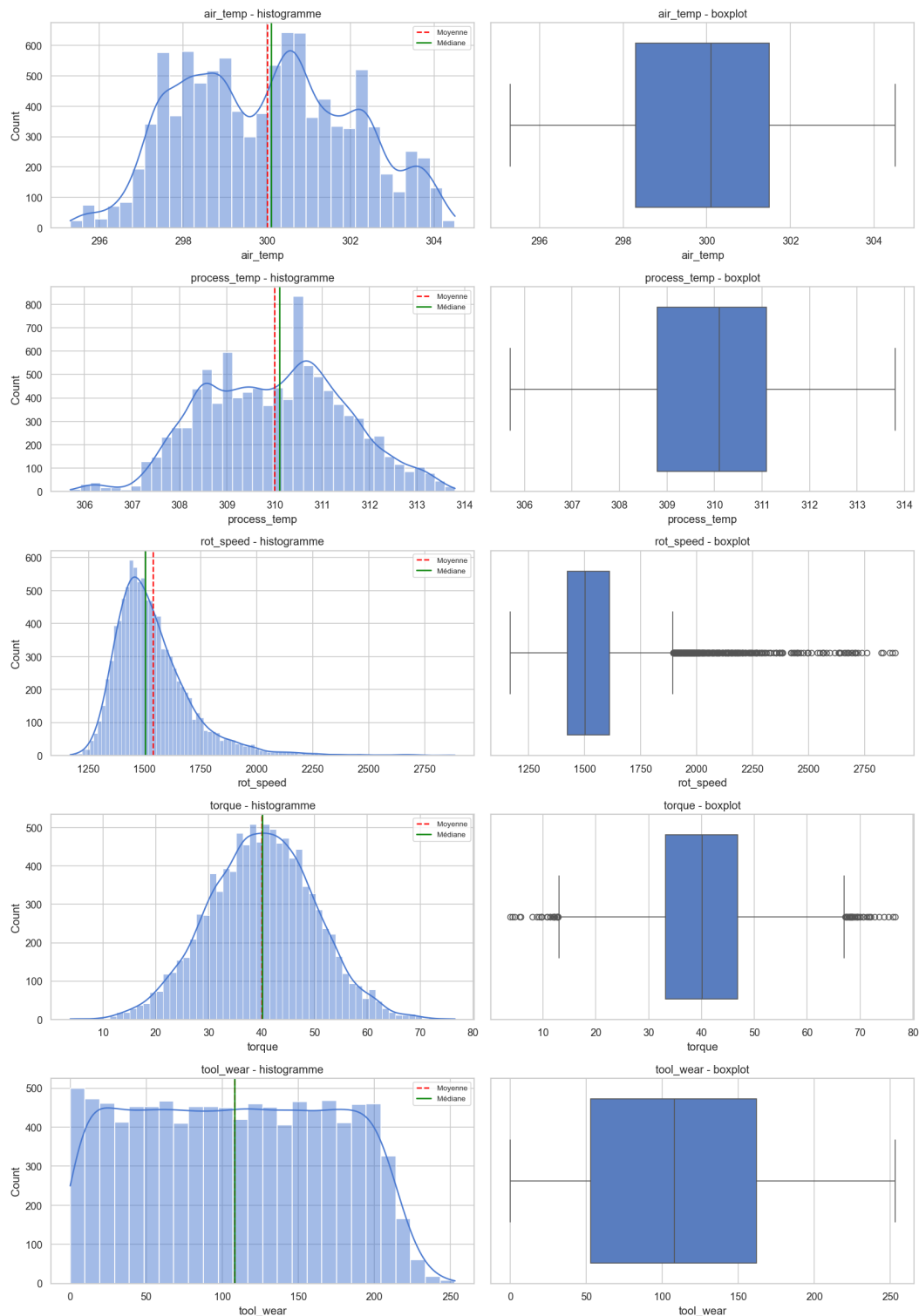


Figure 2: Distributions univariées des features numériques (histogrammes + KDE + boxplots). Ligne rouge : moyenne ; verte : médiane. L'allure plate de `tool_wear` illustre sa distribution quasi-uniforme ; la queue droite de `rot_speed` est visible sur le boxplot (points isolés bien au-delà de Q_3).

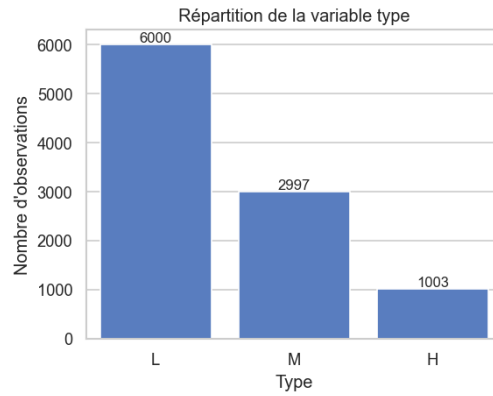


Figure 3: Répartition de la variable qualité `type` (L 60 %, M 30 %, H 10 %). Déséquilibre intentionnel reflétant la répartition industrielle réelle.

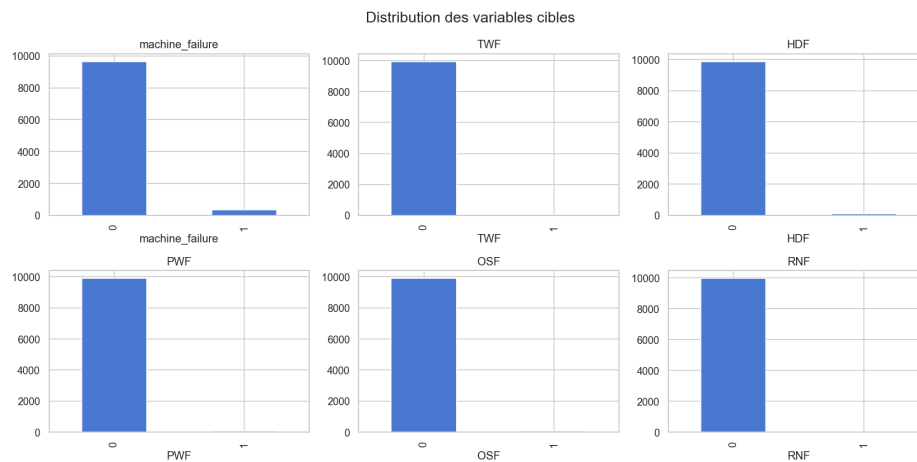


Figure 4: Distribution des six variables cibles. La domination de la classe zéro illustre pourquoi l'accuracy n'est pas une métrique valide ici.

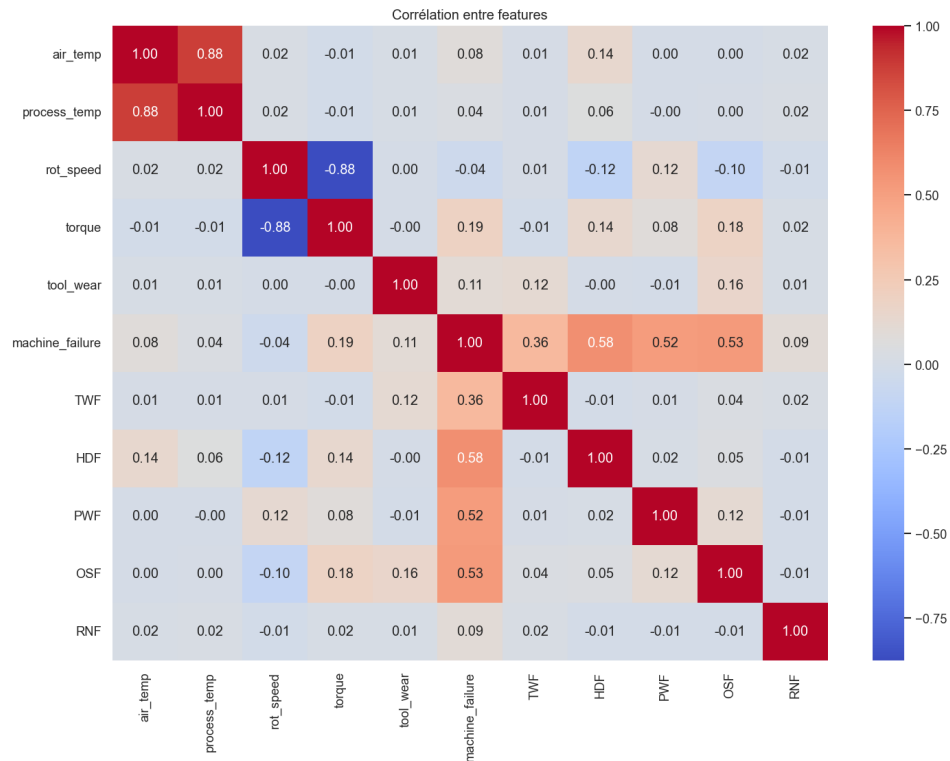


Figure 5: Matrice de corrélation de Pearson. Deux blocs de multicollinéarité : (air_temp, process_temp) avec $r = 0.88$ et (rot_speed, torque) avec $r = -0.88$. Corrélations features $\rightarrow$ cibles toutes inférieures à $|r| = 0.20$.

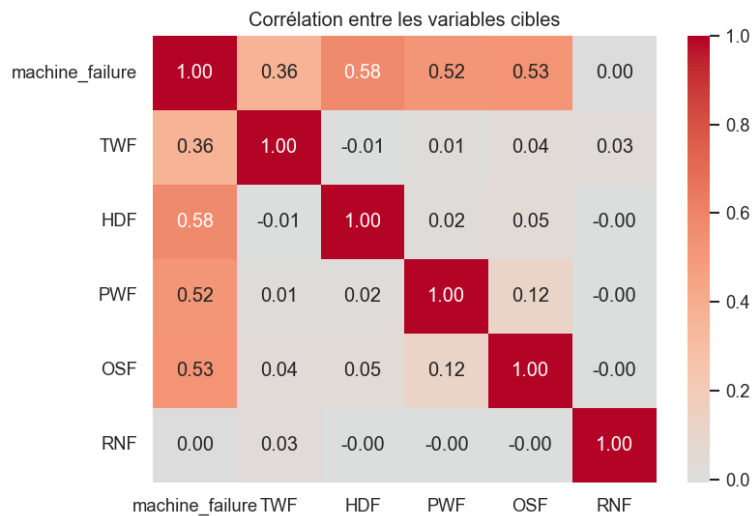


Figure 6: Corrélation entre variables cibles. Quasi-indépendance des sous-pannes ($r < 0.15$) validant leur traitement comme classes mutuellement exclusives.

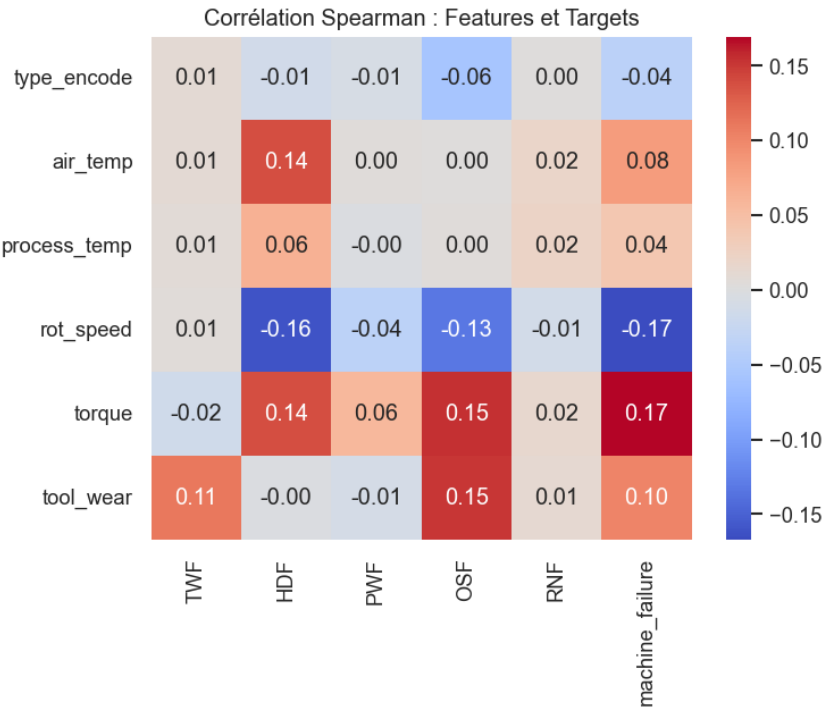


Figure 7: Corrélation de Spearman features → cibles. torque (lié à OSF/PWF) et tool_wear (lié à TWF) sont les plus informatifs. RNF ne corrèle avec aucune feature.

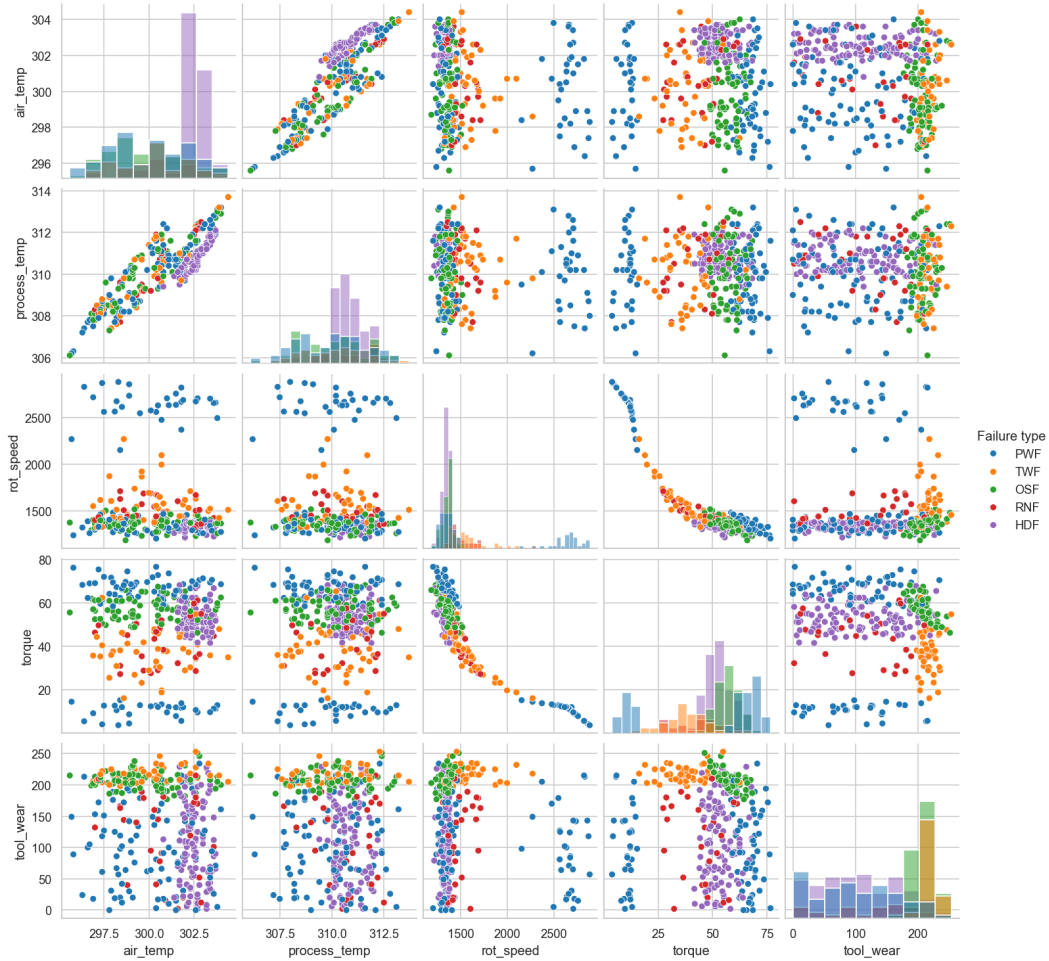


Figure 8: Pairplot coloré par type de panne (observations en défaillance uniquement). L'espace `(rot_speed, torque)` est le plus discriminant : HDF aux températures élevées, OSF à fort couple, PWF aux extrêmes de puissance, RNF diffus sans structure spatiale.

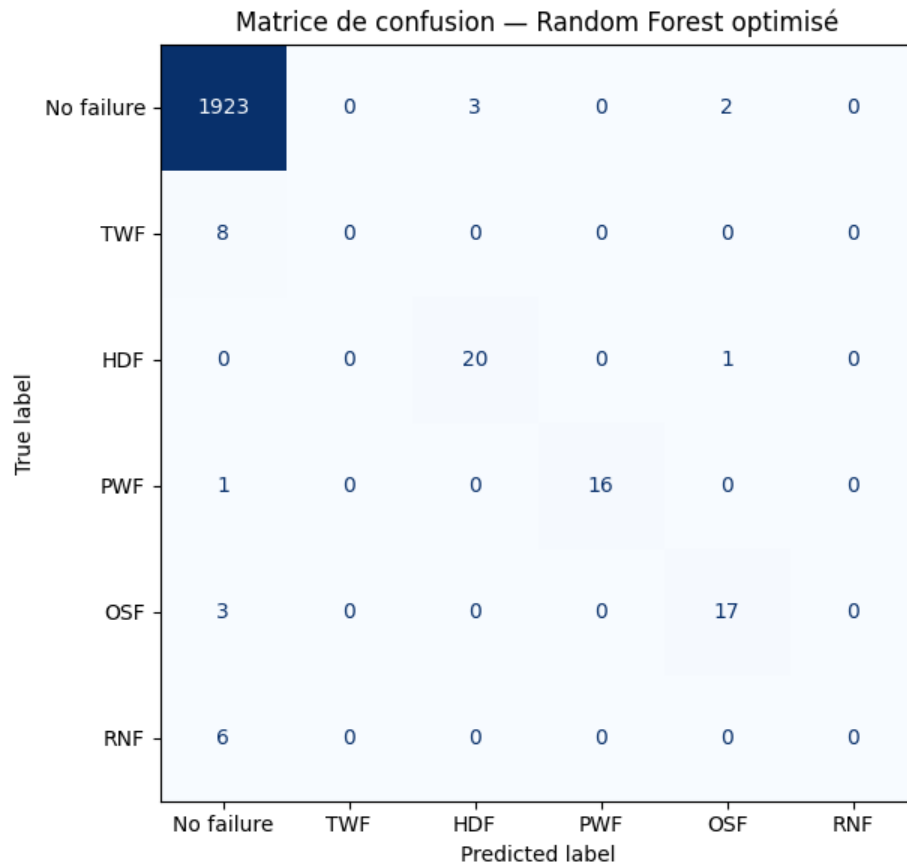


Figure 9: Matrice de confusion (valeurs absolues): Forêt aléatoire, modèle optimisé (MCC 81.7 %, modèle Optuna). Chaque cellule indique le nombre d'observations prédites dans chaque classe. La quasi-totalité des erreurs sont des *faux négatifs* sur les pannes rares : TWF (0/8 détectés) et RNF (0/6) ne sont jamais prédits ; HDF (20/21), PWF (16/17) et OSF (17/20) sont bien reconnus. À comparer à la Figure 1 (vue normalisée, modèle baseline).